

Brightness.Store

Guía de Instalación y Configuración

Instrucciones para levantar el proyecto en un entorno local

Versión 1.0 | Junio 2026

Taller de Programación I — FaCENA, UNNE

1. Requisitos previos

Antes de instalar el proyecto, asegurate de tener instalado lo siguiente en tu sistema:

Software	Versión mínima	Verificar con
PHP	8.1 o superior	<code>php --version</code>
Composer	2.x	<code>composer --version</code>
Node.js	18.x o superior	<code>node --version</code>
NPM	9.x o superior	<code>npm --version</code>
MySQL	8.x	<code>mysql --version</code>
Git	Cualquier versión reciente	<code>git --version</code>

[i] Se recomienda usar Laragon, XAMPP o Herd en Windows para tener PHP y MySQL configurados rápidamente.

2. Clonar el repositorio

Abri una terminal en la carpeta donde querés instalar el proyecto y ejecutá:

```
git clone https://github.com/Tomas-Larrazet/Proyecto-Pagina-Web-Taller-de-Programacion-I-2026.git
cd Proyecto-Pagina-Web-Taller-de-Programacion-I-2026
```

3. Instalar dependencias

3.1. Dependencias PHP (Laravel)

```
composer install
```

3.2. Dependencias JavaScript (frontend)

```
npm install
npm run dev
```

[i] Si solo vas a usarlo en producción, podés usar "npm run build" en lugar de "npm run dev".

4. Configurar el entorno

1 Copiá el archivo de configuración de ejemplo:

```
cp .env.example .env
```

En Windows: `copy .env.example .env`

2 Generá la clave de aplicación:

```
php artisan key:generate
```

3 Editá el archivo `.env` con los datos de tu base de datos:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=brightnesstore
DB_USERNAME=root
DB_PASSWORD=
```

 Cambiá `DB_DATABASE`, `DB_USERNAME` y `DB_PASSWORD` por los valores de tu entorno local. Si usás Laragon, el usuario suele ser root sin contraseña.

5. Crear la base de datos y ejecutar migraciones

5.1. Crear la base de datos

Desde MySQL (ya sea por consola, DBeaver o phpMyAdmin), creá una base de datos vacía con el nombre que pusiste en `DB_DATABASE`:

```
CREATE DATABASE brightnesstore CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

5.2. Ejecutar migraciones y seeders

Este comando crea todas las tablas y las llena con los datos iniciales (categorías, productos y usuarios de prueba):

```
php artisan migrate:fresh --seed
```

 "migrate:fresh" borra todas las tablas existentes antes de crearlas. Si ya tenés datos que querés conservar, usá "php artisan migrate" en su lugar (solo crea las tablas nuevas).

6. Levantar el servidor de desarrollo

```
php artisan serve
```

Una vez ejecutado, abrí tu navegador e ingresá a:

```
http://localhost:8000
```

 Si el puerto 8000 está ocupado, podés usar: `php artisan serve --port=8080`

7. Usuarios de prueba (seeders)

Los seeders crean automáticamente los siguientes usuarios para probar la aplicación:

Nombre	Email	Contraseña	Rol
Administrador	administrador@gmail.com	password	Admin
Cliente Ejemplo	cliente@gmail.com	password	Cliente

[i] Podés ver los usuarios actuales en cualquier momento con: `php artisan tinker → User::select('id','name','email','rol')->get()`

8. Comandos útiles para el desarrollo

Comando	Descripción
<code>php artisan serve</code>	Levanta el servidor de desarrollo
<code>php artisan migrate</code>	Crea las tablas nuevas sin borrar datos
<code>php artisan migrate:fresh --seed</code>	Borra todo, recrea tablas y carga seeders
<code>php artisan db:seed</code>	Ejecuta los seeders sin tocar la estructura
<code>php artisan route:list</code>	Lista todas las rutas registradas
<code>php artisan optimize:clear</code>	Limpia toda la caché de la app
<code>php artisan tinker</code>	Consola interactiva para probar código PHP
<code>npm run dev</code>	Compila assets en modo desarrollo con hot reload
<code>npm run build</code>	Compila assets para producción

9. Estructura de vistas del proyecto

El proyecto incluye las siguientes vistas Blade organizadas en la carpeta `resources/views/`:

Vista	URL	Acceso
-------	-----	--------

Página Principal	/	Todos
Catálogo de productos	/catalogo	Todos
Quiénes Somos	/quienes-somos	Todos
Comercialización	/comercializacion	Todos
Contactos	/contactos	Todos
Términos y Usos	/terminos-y-uso	Todos
Registro de cliente	/registroUsuario	Solo visitantes
Inicio de sesión	/logIn	Solo visitantes
Carrito de compras	/carrito	Solo clientes
Mis compras	/mis-compras	Solo clientes
Mi perfil	/perfil	Solo clientes
Compra exitosa	/compra-exitosa/{id}	Solo clientes
Panel de control	/admin/panel-principal	Solo admins
Gestión de productos	/admin/productos	Solo admins
Historial de ventas	/admin/ventas	Solo admins
Gestión de usuarios	/admin/usuarios	Solo admins
Bandeja de mensajes	/admin/consultas	Solo admins

[i] Las rutas de admin están protegidas por el middleware AdminMiddleware. Las rutas de carrito y perfil requieren autenticación (middleware auth). Las rutas de login y registro solo son accesibles para visitantes (middleware guest).

10. Problemas comunes y soluciones

Error: "No application encryption key has been specified"

Ejecutá:

```
php artisan key:generate
```

Error: "SQLSTATE: Access denied for user"

Las credenciales de la base de datos en el archivo .env son incorrectas. Verificá DB_USERNAME y DB_PASSWORD.

Error: "Class not found" o "Target class does not exist"

Ejecutá:

```
composer dump-autoload  
php artisan optimize:clear
```

Las imágenes de los productos no aparecen

Asegurate de que el enlace simbólico de storage esté creado:

```
php artisan storage:link
```

Los estilos CSS no cargan o están desactualizados

Recompilá los assets:

```
npm run dev
```